

ENGINEERING ASSIGNMENT

Full Stack Developer — Round 2

Assignment: Document Intelligence & Question Extraction Service

Submission Window: 24 Hours

About Pragati Bharati

Pragati Bharti is an AI-guided education and skill-development ecosystem designed to move learning beyond the traditional **Test → Score → Rank** model.

This assignment is intended to evaluate your ability to design and implement production-oriented backend systems while making independent engineering decisions.

Before You Begin

1. Read the complete problem statement before starting.
2. You are expected to make reasonable assumptions where requirements are not explicitly defined.
3. There may be multiple valid approaches to the problem.
4. Clearly document important design decisions and trade-offs.
5. **AI tools are permitted**; however, you are responsible for understanding and validating your submission.
6. The submitted system should be runnable by the evaluation team using the instructions provided.

Problem Statement

Design and implement a scalable Document Processing & Question Extraction Service that accepts PDF documents and images containing examination or question-bank material and converts them into structured, machine-readable questions.

The service should be capable of working with documents originating from different sources and should not depend on a single fixed document layout. A document may contain multiple questions per page, questions spanning multiple pages, different question-numbering formats, embedded images or tables, poor-quality scans, and answer keys located elsewhere in the document.

The final system should expose the extracted questions and associated information through a well-defined API that can be consumed by any downstream examination or assessment platform.

1. Core Requirements

The service should allow an authorized user to:

- Upload a PDF or image.
- Track document-processing status.
- Retrieve extracted questions.
- Retrieve individual question details.
- Retrieve associated answer information where available.
- Identify questions that could not be confidently processed.
- Access the original source/page information for extracted questions.

Document processing should happen asynchronously so that uploading a large document does not require the client to wait for the complete extraction process.

2. Document Processing

The system should support documents containing:

- Digitally generated PDFs.
- Scanned PDFs.
- JPG/JPEG images.
- PNG images.

- Multiple pages.
- Different layouts and formatting styles.

The system should not assume that every document has selectable text or follows a predefined question format.

OCR and/or AI-based document processing may be used where appropriate.

3. Question Extraction

The system should identify individual questions and, where applicable:

- Question number.
- Question text.
- Options.
- Question type.
- Associated images/tables.
- Source document.
- Source page(s).

The extraction should work even when a question continues onto another page.

Different reasonable numbering and option formats should be supported.

The system should preserve the relationship between an extracted question and the page or pages from which it originated.

4. Answer Key

The answer key may:

- Appear at the end of the same document.
- Appear at the beginning.
- Be located on another page.
- Use a different formatting style.
- Be provided as a separate document.

The system should attempt to identify answer-key information and associate it with the corresponding questions.

If an answer cannot be reliably identified, the system should indicate that rather than silently assigning an incorrect answer.

5. Imperfect Documents

The service should be designed to handle realistic input rather than only clean PDFs.

Examples include:

- Blurry or low-resolution scans.
- Rotated pages.
- Inconsistent formatting.
- Missing question numbers.
- Questions split across pages.
- OCR errors.
- Missing or unclear answer keys.
- Images or diagrams within questions.

The system should provide sufficient information to identify extraction results that may require human review.

6. Confidence & Validation

The extracted output should provide an indication of extraction reliability.

The candidate should define an appropriate mechanism for identifying:

- Successfully extracted questions.
- Partially extracted questions.
- Questions requiring review.
- Unmatched or uncertain answers.

The system should preserve enough source information for a reviewer to verify questionable results against the original document.

7. Structured Output

The final questions should be returned in a structured, system-independent format.

A question may contain information such as:

```
{  
  "question": "...",  
  "options": [],  
  "answer": "...",  
  "source_pages": [],  
  "confidence": 0.0  
}
```

The exact schema is left to the candidate.

The output should not be tightly coupled to a particular frontend or examination platform.

8. Multiple Documents

The system should support processing multiple documents and should allow related documents to be associated where appropriate.

For example:

Question Paper.pdf
Answer Key.pdf

may need to be processed together.

The candidate should define how such relationships are represented.

9. Security & File Handling

The service must consider:

- Authentication and authorization.
- File-size limits.
- Supported file types.
- Secure file storage.
- Unauthorized document access.
- Malicious or malformed uploads.
- Protection of external OCR/AI credentials.

Users should only be able to access documents and extracted data they are authorized to access.

10. Technical Constraints

- FastAPI must be used for the API layer.
- PostgreSQL must be used for persistent metadata and extracted structured data.
- Redis should be used where appropriate for asynchronous processing or transient/distributed state.
- PDF and image uploads must be supported.
- Processing must be asynchronous.
- Configuration must be environment-based.
- Secrets must not be committed to the repository.
- APIs should include proper validation and error handling.
- The architecture should support documents being processed concurrently.
- External OCR or AI services may be used.

The candidate is free to choose the specific OCR, AI, parsing, storage, and processing technologies and must document those choices.

11. Required API Surface

At minimum, provide APIs for:

- Uploading documents.
- Checking processing status.
- Retrieving extracted questions.
- Retrieving individual questions.
- Retrieving answer-key information.
- Retrieving extraction warnings/review items.
- Managing related documents where applicable.

Additional endpoints may be introduced where they improve the design.

12. Demonstration Requirements

The candidate should demonstrate the system using multiple documents with different characteristics.

The demonstration should include:

1. Uploading a PDF.
2. Uploading an image.
3. Processing a scanned/low-quality document.
4. Extracting multiple questions.
5. Handling a question spanning multiple pages.
6. Extracting question options.
7. Detecting and associating an answer key.
8. Showing an uncertain/low-confidence extraction.
9. Retrieving the final structured question data.
10. Demonstrating appropriate handling of an invalid or unsupported document.

13. Required Deliverables

Submit:

1. Complete source code.
2. Database migrations/schema.
3. Sample input documents.
4. Sample extracted output.
5. Setup and configuration instructions.
6. Architecture/design documentation.
7. Automated tests.
8. **Postman collection** demonstrating the API workflows.

9. **Swagger UI/OpenAPI documentation** for the API.
10. Demonstration evidence for the required scenarios.

14. Architecture Documentation

Provide a concise explanation of:

- Overall architecture.
- Document-processing approach.
- OCR/AI technology choices.
- Storage design.
- Asynchronous processing.
- Question extraction strategy.
- Answer-key association.
- Confidence/review mechanism.
- Security considerations.
- Scalability considerations.
- Important trade-offs and limitations.

Architecture diagrams are encouraged.

15. AI Usage

AI tools may be used during development.

Candidates may use AI coding assistants, OCR services, document AI, vision models, or other appropriate tools.

The candidate remains responsible for the submitted implementation and must understand and be able to explain the system.

External AI/OCR services used in the solution must be disclosed.

AI-generated code, results, screenshots, or test evidence must not be fabricated or presented without verification.

16. Evaluation Criteria

Category	Weight
Functional correctness	25%
Document & question extraction	20%
Handling of varied/imperfect documents	15%
Architecture & scalability	15%
Answer-key association & validation	10%

Category	Weight
Security & error handling	5%
Testing	5%
Code quality & documentation	5%

Final Requirement

The candidate is **not expected to build a perfect OCR or document-understanding system.**

The objective is to demonstrate the ability to build a reliable engineering system around unstructured and imperfect input, make appropriate technology choices, handle uncertainty, and produce useful structured output.

Candidates are free to choose implementation details where the requirements do not prescribe a specific solution.